Task 4: Production Readiness

Scaling

The API deployment was scaled from 2 replicas to 3 replicas:

```
kubectl scale deployment product-catalog-api --replicas=3 -n product-catalog
kubectl rollout status deployment/product-catalog-api -n product-catalog --
timeout=180s
kubectl get pods -n product-catalog -o wide
kubectl get deployment product-catalog-api -n product-catalog -o wide
```

Result:

```
deployment.apps/product-catalog-api scaled
deployment "product-catalog-api" successfully rolled out
```

The deployment status shows that all 3 API replicas are available:

```
NAME                READY   UP-TO-DATE   AVAILABLE
product-catalog-api   3/3     3            3
```

The pod list shows three running API pods:

```
product-catalog-api-77c8d8bb78-fwzl8   1/1   Running
product-catalog-api-77c8d8bb78-pxpvp   1/1   Running
product-catalog-api-77c8d8bb78-v278t   1/1   Running
```

Health Checks

The API deployment defines both a readiness probe and a liveness probe:

```
readinessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10

livenessProbe:
  httpGet:
```

```
    path: /health
    port: 8080
  initialDelaySeconds: 15
  periodSeconds: 20
```

Readiness vs. liveness

A readiness probe tells Kubernetes whether a container is ready to receive traffic. If the readiness probe succeeds, the pod can be used as an endpoint behind the service. If it fails, Kubernetes keeps the pod running but removes it from service load balancing.

A liveness probe tells Kubernetes whether a container is still healthy enough to keep running. If the liveness probe fails repeatedly, Kubernetes restarts the container.

What happens when the readiness probe fails?

If the readiness probe fails, the pod becomes `NotReady`. The pod is not killed. Kubernetes removes it from the service endpoints, so new traffic is not routed to that pod until the readiness probe succeeds again.

This is useful during startup or temporary dependency problems. For example, if the API cannot reach PostgreSQL yet, it should not receive user traffic, but it may recover without a restart.

What happens when the liveness probe fails?

If the liveness probe fails repeatedly, Kubernetes treats the container as unhealthy and restarts it. This is useful when the process is stuck or in a broken state that will not recover by itself.

For this API, the liveness probe checks `/health` on port `8080`. If the API stops responding to this endpoint, Kubernetes can restart the container.

Why different `initialDelaySeconds` values?

The readiness probe has a shorter initial delay (`5s`) because Kubernetes should quickly know when the pod can receive traffic.

The liveness probe has a longer initial delay (`15s`) because the application needs time to start before Kubernetes begins restart decisions. Starting liveness checks too early can cause unnecessary restarts during normal startup.

Using different delays prevents the pod from receiving traffic too early while also avoiding premature restarts.

Resource Requests and Limits

The API deployment defines CPU and memory requests and limits:

```
resources:
  requests:
    memory: "64Mi"
    cpu: "100m"
  limits:
```

```
      memory: "128Mi"
      cpu: "250m"
```

What happens if the memory limit is exceeded?

If a container exceeds its memory limit, Kubernetes can terminate it with an out-of-memory condition. The pod status may show `OOMKilled`. Depending on the restart policy, Kubernetes then restarts the container.

Memory is a hard limit. The container cannot keep using memory beyond the configured limit.

What happens if the CPU limit is exceeded?

If a container tries to use more CPU than its configured limit, Kubernetes throttles it. The container is not usually killed just because it exceeds CPU. Instead, it runs slower because it cannot consume more CPU than the limit allows.

CPU is therefore enforced differently from memory: CPU is throttled, memory can lead to termination.

Why specify both requests and limits?

Requests and limits solve different problems.

Requests tell Kubernetes how much CPU and memory the container is expected to need. The scheduler uses requests to decide which node has enough capacity for the pod.

Limits define the maximum amount of CPU and memory the container may use. They protect the node and other workloads from one container consuming too many resources.

Using both gives Kubernetes enough information to schedule pods predictably and also enforces boundaries at runtime.

The `describe deployment` output shows:

- `Replicas: 3 desired | 3 updated | 3 total | 3 available`
- readiness probe configuration
- liveness probe configuration
- CPU and memory requests/limits